Module Guide for RealmForge

Team 20, Aeolus Software
Andy Huynh
Andreas Sideris
Humna Saeed
Christopher Ludwig
Owen Johnson

November 13, 2026

1 Revision History

Date	Version	Notes
Date 1	1.0	Notes
Date 2	1.1	Notes

2 Reference Material

This section records information for easy reference.

2.1 Abbreviations and Acronyms

symbol	description
AC	Anticipated Change
DAG	Directed Acyclic Graph
M	Module
MG	Module Guide
OS	Operating System
R	Requirement
SC	Scientific Computing
SRS	Software Requirements Specification
RealmForge	Explanation of program name
UC	Unlikely Change
[etc. —SS]	[... —SS]

Contents

1 Revision History **i**

2 Reference Material **ii**
2.1 Abbreviations and Acronyms ii

3 Introduction **1**

4 Anticipated and Unlikely Changes **2**
4.1 Anticipated Changes 2
4.2 Unlikely Changes 2

5 Module Hierarchy **2**

6 Connection Between Requirements and Design **4**

7 Module Decomposition **4**
7.1 Hardware Hiding Modules 4
7.1.1 User Input Module M1 4
7.2 Behaviour-Hiding Module 4
7.2.1 Configuration Interface Module M4 4
7.2.2 Variation Configuration Module M5 5
7.2.3 Enemy Character Module M11 5
7.3 Software Decision Module 5

8 Traceability Matrix **5**

9 Use Hierarchy Between Modules **7**

10 User Interfaces **8**

11 Design of Communication Protocols **8**

12 Database Design **8**

13 Timeline **8**

14 Generative AI Use Disclosure **8**
14.1 AI Tools Used 8
14.2 How and Where Used 8
14.3 Verification of AI-Generated Content 9

List of Tables

1 Module Hierarchy 3
2 Trace Between Requirements and Modules 6
3 Trace Between Anticipated Changes and Modules 7

List of Figures

1 Use hierarchy among modules 7

3 Introduction

Decomposing a system into modules is a commonly accepted approach to developing software. A module is a work assignment for a programmer or programming team (Parnas et al., 1984). We advocate a decomposition based on the principle of information hiding (Parnas, 1972). This principle supports design for change, because the "secrets" that each module hides represent likely future changes. Design for change is valuable in SC, where modifications are frequent, especially during initial development as the solution space is explored.

Our design follows the rules layed out by Parnas et al. (1984), as follows:

- System details that are likely to change independently should be the secrets of separate modules.
- Each data structure is implemented in only one module.
- Any other program that requires information stored in a module's data structures must obtain it by calling access programs belonging to that module.

After completing the first stage of the design, the Software Requirements Specification (SRS), the Module Guide (MG) is developed (Parnas et al., 1984). The MG specifies the modular structure of the system and is intended to allow both designers and maintainers to easily identify the parts of the software. The potential readers of this document are as follows:

- New project members: This document can be a guide for a new project member to easily understand the overall structure and quickly find the relevant modules they are searching for.
- Maintainers: The hierarchical structure of the module guide improves the maintainers' understanding when they need to make changes to the system. It is important for a maintainer to update the relevant sections of the document after changes have been made.
- Designers: Once the module guide has been written, it can be used to check for consistency, feasibility, and flexibility. Designers can verify the system in various ways, such as consistency among modules, feasibility of the decomposition, and flexibility of the design.

The rest of the document is organized as follows. Section 4 lists the anticipated and unlikely changes of the software requirements. Section 5 summarizes the module decomposition that was constructed according to the likely changes. Section 6 specifies the connections between the software requirements and the modules. Section 7 gives a detailed description of the modules. Section 8 includes two traceability matrices. One checks the completeness of the design against the requirements provided in the SRS. The other shows the relation between anticipated changes and the modules. Section 9 describes the use relation between modules.

4 Anticipated and Unlikely Changes

This section lists possible changes to the system. According to the likeliness of the change, the possible changes are classified into two categories. Anticipated changes are listed in Section 4.1, and unlikely changes are listed in Section 4.2.

4.1 Anticipated Changes

Anticipated changes are the source of the information that is to be hidden inside the modules. Ideally, changing one of the anticipated changes will only require changing the one module that hides the associated decision. The approach adapted here is called design for change.

AC1: The specific hardware on which the software is running.

AC2: The format of the initial input data.

...

[Anticipated changes relate to changes that would be made in requirements, design or implementation choices. They are not related to changes that are made at run-time, like the values of parameters. —SS]

4.2 Unlikely Changes

The module design should be as general as possible. However, a general system is more complex. Sometimes this complexity is not necessary. Fixing some design decisions at the system architecture stage can simplify the software design. If these decision should later need to be changed, then many parts of the design will potentially need to be modified. Hence, it is not intended that these decisions will be changed.

UC1: Input/Output devices (Input: File and/or Keyboard, Output: File, Memory, and/or Screen).

...

5 Module Hierarchy

This section provides an overview of the module design. Modules are summarized in a hierarchy decomposed by secrets in Table 1. The modules listed below, which are leaves in the hierarchy tree, are the modules that will actually be implemented.

M1: User Input Module

M2: Save Data IO Module

M3: Main Menu Interface Module

M4: Configuration Interface Module

M5: Variation Configuration Module

M6: Game World Interface Module

M7: Game World Module

M8: Combat Encounter Interface Module

M9: Combat Encounter Module

M10: Player Character Module

M11: Enemy Character Module

M12: Central Hub Interface Module

M13: Central Hub Module

M14: Save Data Storage Module

Level 1	**Level 2**
Hardware-Hiding Module	M1: User Input Module M2: Save Data IO Module
Behaviour-Hiding Module	M3: Main Menu Interface Module M4: Configuration Interface Module M5: Variation Configuration Module M6: Game World Interface Module M7: Game World Module M8: Combat Encounter Interface Module M9: Combat Encounter Module M10: Player Character Module M11: Enemy Character Module M12: Central Hub Interface Module M13: Central Hub Module
Software Decision Module	M14: Save Data Storage Module

Table 1: Module Hierarchy

6 Connection Between Requirements and Design

The design of the system is intended to satisfy the requirements developed in the SRS. In this stage, the system is decomposed into modules. The connection between requirements and modules is listed in Table 2.

[The intention of this section is to document decisions that are made "between" the requirements and the design. To satisfy some requirements, design decisions need to be made. Rather than make these decisions implicit, they are explicitly recorded here. For instance, if a program has security requirements, a specific design decision may be made to satisfy those requirements with a password. —SS]

7 Module Decomposition

Modules are decomposed according to the principle of "information hiding" proposed by Parnas et al. (1984). The *Secrets* field in a module decomposition is a brief statement of the design decision hidden by the module. The *Services* field specifies *what* the module will do without documenting *how* to do it. For each module, a suggestion for the implementing software is given under the *Implemented By* title. If the entry is *OS*, this means that the module is provided by the operating system or by standard programming language libraries. *RealmForge* means the module will be implemented by the RealmForge software. The software is implemented using the Godot game engine. If a module's services are provided by Godot's capabilities, the entry *Godot Engine* signifies that the module is implemented by Godot.

Only the leaf modules in the hierarchy have to be implemented. If a dash (–) is shown, this means that the module is not a leaf and will not have to be implemented.

7.1 Hardware Hiding Modules

7.1.1 User Input Module M1

Secrets: The details of how user input is received from the user's input device (a controller, or keyboard and mouse), through the user's machine.

Services: Serves as an abstraction of the physical input provided to the system by the user. This module receives input from the user's input device, and provides the input to other system modules, irrespective of how the input is physically provided.

Implemented By: Godot Engine

7.2 Behaviour-Hiding Module

7.2.1 Configuration Interface Module M4

Secrets: The programmed logic used to provide a graphical interface to the user to allow them to configure the system, as managed by the Variation Configuration Module M5.

Services: Provides a graphical user interface to configure the system at its explicit variation points. Selections made using this interface are then provided to other system modules, to facilitate the behaviour of the system irrespective of how the interface is presented or constructed.

Implemented By: RealmForge

7.2.2 Variation Configuration Module M5

Secrets: The programmed logic used to manage configurations of the system's variation points.

Services: Provides input to other system modules to configure their features with respect to the user's selected variation point configurations (irrespective of how these configurations are managed). These configurations can take the form of discrete variables, or selections of larger structural features (as discussed in the project's Feature-Oriented Domain Analysis Report).

Implemented By: RealmForge

7.2.3 Enemy Character Module M11

Secrets: An enemy character in the RealmForge game's internal state.

Services: Provides the state of an enemy character to other system modules where needed. Provides the enemy character's displayed image and animation to the Godot Engine for display, and provides their health, game actions, and other required attributes to the Combat Encounter Module M9 to facilitate combat encounter game scenes.

Implemented By: RealmForge

7.3 Software Decision Module

8 Traceability Matrix

This section shows two traceability matrices: between the modules and the requirements and between the modules and the anticipated changes.

Req.	**Modules**
ENV-002	M1
ENV-003	M4, M5
SFR-001	M1, M4, M5
SFR-002	M4
SFR-003	M5
SFR-004	M1, M4, M5
SFR-005	M5
SFR-006	M4, M5
SFR-007	M4, M5
SFR-008	M4, M5
SFR-009	M4, M5
SFR-010	M4
SFR-011	M11
SFR-012	M11
SFR-013	M11
SFR-014	M11
SFR-015	M11
SFR-017	M11
SNFR-002	M11
SNFR-003	M4, M5
SNFR-006	M11
SNFR-007	M1
SNFR-008	M4
SNFR-010	M4
SNFR-011	M1

Table 2: Trace Between Requirements and Modules

AC	Modules
AC1	M??
AC2	M1
AC??	M??
AC??	M??
AC??	M??
AC??	M??
AC??	M??
AC??	M??
AC??	M??
AC??	M??
AC??	M??
AC??	M??

Table 3: Trace Between Anticipated Changes and Modules

9 Use Hierarchy Between Modules

In this section, the uses hierarchy between modules is provided. Parnas (1978) said of two programs A and B that A *uses* B if correct execution of B may be necessary for A to complete the task described in its specification. That is, A *uses* B if there exist situations in which the correct functioning of A depends upon the availability of a correct implementation of B. Figure 1 illustrates the use relation between the modules. It can be seen that the graph is a directed acyclic graph (DAG). Each level of the hierarchy offers a testable and usable subset of the system, and modules in the higher level of the hierarchy are essentially simpler because they use modules from the lower levels.

[The uses relation is not a data flow diagram. In the code there will often be an import statement in module A when it directly uses module B. Module B provides the services that module A needs. The code for module A needs to be able to see these services (hence the import statement). Since the uses relation is transitive, there is a use relation without an import, but the arrows in the diagram typically correspond to the presence of import statement. —SS]

[If module A uses module B, the arrow is directed from A to B. —SS]

Figure 1: Use hierarchy among modules

10 User Interfaces

[Design of user interface for software and hardware. Attach an appendix if needed. Drawings, Sketches, Figma —SS]

11 Design of Communication Protocols

[If there is communication between hardware and software, it may be necessary to explain the design of the communication protocol. This section should be filled in with NA if it is not appropriate for a given project. —SS]

12 Database Design

[If a database is a significant component of the project, the database design should be summarized here. This section should be filled in with NA if it is not appropriate for a given project. —SS]

13 Timeline

[Schedule of tasks and who is responsible —SS]

[You can point to GitHub if this information is included there —SS]

14 Generative AI Use Disclosure

14.1 AI Tools Used

- Anthropic Claude Sonnet 5.5
- Google Search AI Overview (Google Gemini)

14.2 How and Where Used

AI tools were used for assistance with LaTeX syntax and commands, during the development of the Module Hierarchy, and portions of the Module Decomposition section. AI tools were used for guidance on how to correctly structure LaTeX code to create a hyperlink-able list of software modules and include them in the module hierarchy table, as well as other various small formatting fixes. In these sections, AI tools were not used to generate any content text, or wholly generate any of the ideas or conclusions presented.

14.3 Verification of AI-Generated Content

Any LaTeX syntax or code provided by AI tools was verified by testing compilation in LaTeX documents, and ensuring the resulting output was correctly formatted and matched the needs of the document. No ideas or text were newly generated by AI tools and presented in the document.

References

David L. Parnas. On the criteria to be used in decomposing systems into modules. *Comm. ACM*, 15(2):1053–1058, December 1972.

David L. Parnas. Designing software for ease of extension and contraction. In *ICSE '78: Proceedings of the 3rd international conference on Software engineering*, pages 264–277, Piscataway, NJ, USA, 1978. IEEE Press. ISBN none.

D.L. Parnas, P.C. Clement, and D. M. Weiss. The modular structure of complex systems. In *International Conference on Software Engineering*, pages 408–419, 1984.